Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Tutorial

Week 6

Question 1

There exists a comparison sort of 5 numbers that uses at most 6 comparisons in the worst case.

- True
- False

Question 1

Answer: False

Number of leaves in a decision tree which sorts 5 numbers: 5! = 120.

With 6 comparisons, we have up to 2^6 possible outputs, which is fewer than 120, hence cannot produce 120 leaves.

Question 2

Consider an array A of n elements. Which of following statement is **incorrect** (in the comparison model)?

- We can make a heap from A in $O(n)$ time.
- We can traverse a tree binary search tree built from A in $O(n)$ time.
- We can make a balanced binary search tree T from A in $O(n)$ time.
- We can find a k-th smallest element of A in sub-quadratic time

Question 2

Answer: C is false.

Consider the following algorithm for **sorting**:

1. Construct a balanced tree in O(n) time??
2. Do an in order traversal of the tree, known to be possible in O(n) time.

If the time for constructing a balanced tree is O(n), then the total time for sorting using the algorithm would be O(n), contradicting the comparison sort lower bound.

Question 2

A is true. A heap can be constructed in linear time. Discuss offline if you do not know how.

B is true. Tree traversal can be done in linear time as it passes through each element a constant number of times.

D is true. Use worst case linear time select algorithm (covered in lecture).

Question 3

There is a list of records of n students, already sorted by students' name. The final exam score of a student is an integer between 0 and 100. We now want to sort this list lexicographically, first by score, then by name. Which is the following is asymptotically the fastest algorithm that will produce the lexicographically sorted records when used to sort the records by score?

- Merge sort
- Counting sort
- Quick sort
- Insertion sort

Question 3

Answer: B Counting sort

Run counting sort on the exam score column, moving the whole record during the sort. Stability ensures that the names remain sorted within the same score. Runtime is $\Theta(n+k)$ where k is 100.

The other algorithms are all $\Theta(n \log n)$ worst case or expected case.

Question 4

A set of n integers in the range $\{1, 2, \ldots, n\}$ can be sorted by RADIX-SORT in $O(n)$ time by running COUNTING-SORT on each bit of the binary representation. (Hint: how many bits does it take to represent numbers in $\{1, 2, \ldots, n\}$?)

- True
- False

Question 4

Answer: False

To represent numbers in {1, …, n} we need $\Theta(\lg n)$ bits.

Each iteration of counting sort takes $\Theta(n)$ time.

If we run counting sort on each bit, we will run $\Theta(\lg n)$ iterations, giving a total runtime of $\Theta(n \lg n)$.

What is the right digit size to get $\Theta(n)$ runtime for radix sort in this case?